```
    public List<City> Cities { get; set; } = new List<City>();  ❸
}
```

❶ Foreign key property

❷ Reference navigation property representing the one country a city belongs to

❸ Collection navigation property representing the many cities that can belong to a country, instantiated to ensure that it can never be null

Object-oriented purists are generally not keen on the idea of including foreign key properties in domain classes because they see this as an example of the relational database world "bleeding" into the domain. If you omit the foreign key property, EF Core will create a *shadow foreign key* property (http://mng.bz/Xawa) as part of its conceptual model.

By convention, a property named `Id` or `<entity_name>Id` is seen as the primary key value, and EF Core will map it to an `IDENTITY` column. Both of your domain classes have an `Id` property, but EF Core will also see `CityId` in the city class or `CountryId` in the country class as representing the primary key. Foreign key properties should use the name of the reference navigation property followed by `Id`—`CountryId`, in our example.

You have yet to create a class in your model representing properties vacationers can rent. So using the information above to create a one-to-many relationship between cities and properties, the following listing shows the `Property` class that should be added to the Models folder.

Listing 8.3 The `Property` class

```
public class Property
{
    public int Id { get; set; }                    ❶
    public string Name { get; set; }
    public string Address { get; set; }
    public int CityId { get; set; }                ❷
```